

FRD

Functional Requirements Document (FRD)

AWS Glue ETL Pipeline for Customer Order Processing

-

1. Document Overview

Purpose

This Functional Requirements Document defines the functional behavior of an AWS Glue-based ETL pipeline that ingests customer and order data from S3, performs data quality operations, implements SCD Type 2 logic using Apache Hudi, and produces analytical aggregates for business intelligence purposes.

Scope

- In Scope:
 - Ingestion of customer and order CSV files from S3
 - Data cleaning (null removal, deduplication)
 - Registration of datasets in AWS Glue Data Catalog
 - SCD Type 2 implementation for order summary using Hudi format
 - Incremental processing triggered by customer updates
 - Generation of customer aggregate spend analytics table
 - Storage of curated and analytics data in S3
- Out of Scope:
 - Real-time streaming ingestion
 - Data masking or encryption logic
 - User interface or reporting layer
 - Alerting or notification mechanisms
 - Data retention policies

Assumptions

1. CSV files in source S3 paths are well-formed and use standard delimiters
2. CustId is the primary key for Customer data
3. OrderId is the primary key for Order data
4. The join between Customer and Order is performed on CustId
5. "Null" (string) and NULL (actual null) are the only null representations to handle
6. Duplicate detection is based on all columns in each dataset

7. SCD Type 2 change detection is based on any column change in the joined customer-order record
8. Date fields are in a parseable format (exact format not specified in BRD)
9. AWS Glue job has appropriate IAM permissions for S3, Glue Catalog, and CloudWatch
10. Hudi configuration parameters (precombine key, record key, partition) will be defined in TRD
11. Incremental processing logic assumes availability of change data or full refresh comparison
-

2. Functional Requirements

FR-INGEST-001: Ingest Customer Data from S3

- Title: Load Customer CSV Data from S3

- Description:

The system shall read customer data in CSV format from the specified S3 location and load it into a structured dataset for processing.

- Business Objective:

Enable downstream processing by making customer master data available in the ETL pipeline.

- Inputs:

- Source: s3://adif-sdlc/sdlc_wizard/customerdata/
- Format: CSV
- Schema:
- CustId: (datatype not specified)
- Name: (datatype not specified)
- EmailId: (datatype not specified)
- Region: (datatype not specified)

- Processing Rules / Functional Steps:

1. Read all CSV files from the source S3 path
2. Parse CSV content using appropriate delimiter and header detection
3. Map columns to the defined schema: CustId, Name, EmailId, Region
4. Load data into a DataFrame or equivalent structure for further processing

- Outputs:

- In-memory customer dataset ready for cleaning

- Preconditions:

- Source S3 path exists and contains at least one CSV file

- AWS Glue job has read permissions on the S3 bucket
- Postconditions:
- Customer data is loaded and accessible for subsequent ETL steps
- Error / Validation Conditions:
- If source path does not exist, job must fail with clear error message
- If CSV parsing fails, job must log error and fail
- If schema columns are missing, job must fail with schema mismatch error
-

FR-INGEST-002: Ingest Order Data from S3

- Title: Load Order CSV Data from S3
- Description:

The system shall read order transaction data in CSV format from the specified S3 location and load it into a structured dataset for processing.

- Business Objective:

Enable order transaction analysis by making order data available in the ETL pipeline.

- Inputs:
- Source: s3://adif-sdlc/sdlc_wizard/orderdata/
- Format: CSV
- Schema:
- OrderId: (datatype not specified)
- ItemName: (datatype not specified)
- PricePerUnit: (datatype not specified)
- Qty: (datatype not specified)
- Date: (datatype not specified)
- CustId: (datatype not specified)
- Processing Rules / Functional Steps:
- 1. Read all CSV files from the source S3 path
- 2. Parse CSV content using appropriate delimiter and header detection
- 3. Map columns to the defined schema: OrderId, ItemName, PricePerUnit, Qty, Date, CustId
- 4. Load data into a DataFrame or equivalent structure for further processing
- Outputs:

- In-memory order dataset ready for cleaning
- Preconditions:
 - Source S3 path exists and contains at least one CSV file
 - AWS Glue job has read permissions on the S3 bucket
- Postconditions:
 - Order data is loaded and accessible for subsequent ETL steps
- Error / Validation Conditions:
 - If source path does not exist, job must fail with clear error message
 - If CSV parsing fails, job must log error and fail
 - If schema columns are missing, job must fail with schema mismatch error
-

FR-CLEAN-001: Remove Null Values from Customer and Order Data

- Title: Data Quality - Null Value Removal

- Description:

The system shall remove all records containing "Null" (string literal) or NULL (actual null value) in any column from both customer and order datasets.

- Business Objective:

Ensure data quality by eliminating incomplete records that could cause downstream processing errors or analytical inaccuracies.

- Inputs:

- Raw customer dataset (from FR-INGEST-001)
- Raw order dataset (from FR-INGEST-002)

- Processing Rules / Functional Steps:

1. For customer dataset:

- Identify rows where any column contains the string "Null" or actual NULL value
- Remove identified rows from the dataset

2. For order dataset:

- Identify rows where any column contains the string "Null" or actual NULL value
- Remove identified rows from the dataset

3. Log count of removed records for each dataset

- Outputs:

- Cleaned customer dataset (null-free)
- Cleaned order dataset (null-free)
- Preconditions:
 - Customer and order datasets are loaded in memory
- Postconditions:
 - No record in either dataset contains "Null" or NULL in any column
 - Record counts are logged for audit purposes
- Error / Validation Conditions:
 - If all records are removed, log warning but continue processing with empty dataset
-

FR-CLEAN-002: Remove Duplicate Records from Customer and Order Data

- Title: Data Quality - Duplicate Record Removal

- Description:

The system shall identify and remove duplicate records from both customer and order datasets based on all column values.

- Business Objective:

Prevent duplicate processing and ensure data integrity by maintaining only unique records in each dataset.

- Inputs:
 - Null-cleaned customer dataset (from FR-CLEAN-001)
 - Null-cleaned order dataset (from FR-CLEAN-001)

- Processing Rules / Functional Steps:

1. For customer dataset:

- Identify duplicate rows where all columns (CustId, Name, EmailId, Region) match
- Retain only the first occurrence of each duplicate set
- Remove subsequent duplicates

2. For order dataset:

- Identify duplicate rows where all columns (OrderId, ItemName, PricePerUnit, Qty, Date, CustId) match
- Retain only the first occurrence of each duplicate set
- Remove subsequent duplicates

3. Log count of removed duplicates for each dataset

- Outputs:
- Deduplicated customer dataset
- Deduplicated order dataset
- Preconditions:
- Null removal (FR-CLEAN-001) is complete
- Postconditions:
- Each dataset contains only unique records
- Duplicate counts are logged for audit purposes
- Error / Validation Conditions:
- If all records are duplicates except one, log warning but continue with single record
-

FR-CATALOG-001: Register Customer Data in Glue Data Catalog

- Title: Catalog Registration - Customer Table
- Description:

The system shall register the cleaned customer dataset as a table in the AWS Glue Data Catalog for query and governance purposes.

- Business Objective:

Enable SQL-based querying via Athena and provide metadata management for the customer dataset.

- Inputs:
- Deduplicated customer dataset (from FR-CLEAN-002)
- Processing Rules / Functional Steps:
 1. Write cleaned customer data to S3 in a queryable format (Parquet or similar)
 2. Register table in Glue Data Catalog with the following metadata:
 - Database: gen_ai_poc_databrickscoe
 - Table Name: sdlc_wizard_customer
 - Schema: CustId, Name, EmailId, Region
 3. Update catalog with S3 location and partition information (if applicable)
- Outputs:
- Glue Catalog table: gen_ai_poc_databrickscoe.sdlc_wizard_customer
- Customer data persisted in S3

- Preconditions:
- Glue database "gen_ai_poc_databricksco" exists
- AWS Glue job has permissions to create/update catalog tables
- Postconditions:
- Table is queryable via Athena
- Metadata is available in Glue Data Catalog
- Error / Validation Conditions:
- If database does not exist, job must fail with clear error
- If table creation fails, job must fail and log error details
-

FR-CATALOG-002: Register Order Data in Glue Data Catalog

- Title: Catalog Registration - Order Table
- Description:

The system shall register the cleaned order dataset as a table in the AWS Glue Data Catalog for query and governance purposes.

- Business Objective:

Enable SQL-based querying via Athena and provide metadata management for the order dataset.

- Inputs:
- Deduplicated order dataset (from FR-CLEAN-002)
- Processing Rules / Functional Steps:
 1. Write cleaned order data to S3 in a queryable format (Parquet or similar)
 2. Register table in Glue Data Catalog with the following metadata:
 - Database: gen_ai_poc_databricksco
 - Table Name: sdlc_wizard_order
 - Schema: OrderId, ItemName, PricePerUnit, Qty, Date, CustId
 3. Update catalog with S3 location and partition information (if applicable)
- Outputs:
- Glue Catalog table: gen_ai_poc_databricksco.sdlc_wizard_order
- Order data persisted in S3
- Preconditions:
- Glue database "gen_ai_poc_databricksco" exists

- AWS Glue job has permissions to create/update catalog tables
- Postconditions:
- Table is queryable via Athena
- Metadata is available in Glue Data Catalog
- Error / Validation Conditions:
- If database does not exist, job must fail with clear error
- If table creation fails, job must fail and log error details
-

FR-SCD2-001: Implement SCD Type 2 for Order Summary

- Title: Slowly Changing Dimension Type 2 - Order Summary Table
- Description:

The system shall join customer and order data, then implement SCD Type 2 logic to track historical changes in the ordersummary table using Apache Hudi format on S3.

- Business Objective:

Maintain complete historical record of order summaries with customer information, enabling time-based analysis and audit trails.

- Inputs:
- Cleaned customer dataset (from FR-CLEAN-002)
- Cleaned order dataset (from FR-CLEAN-002)
- Existing ordersummary table (if present) from s3://adif-sdlc/curated/sdlc_wizard/ordersummary/
- Processing Rules / Functional Steps:
- 1. Join customer and order datasets on CustId
- 2. For each joined record, compare with existing ordersummary table:
- If record is new (does not exist in target):
- Insert as new active row with IsActive=true, StartDate=current timestamp, EndDate=null, OpTs=current timestamp
- If record exists and any column value has changed:
- Update existing row: set IsActive=false, EndDate=current timestamp, OpTs=current timestamp
- Insert new row: set IsActive=true, StartDate=current timestamp, EndDate=null, OpTs=current timestamp
- If record exists and no change detected:
- No action required

3. Write results to S3 using Hudi format

4. Update Glue Catalog table: gen_ai_poc_databricksco.ordersummary

- Outputs:
- Hudi table at s3://adif-sdlc/curated/sdlc_wizard/ordersummary/
- Glue Catalog table: gen_ai_poc_databricksco.ordersummary
- SCD Type 2 columns: IsActive, StartDate, EndDate, OpTs
- Preconditions:
- Customer and order data are cleaned and deduplicated
- S3 target path is accessible
- Hudi libraries are available in Glue environment
- Postconditions:
- All current records have IsActive=true and EndDate=null
- All superseded records have IsActive=false and EndDate populated
- Historical lineage is preserved
- Error / Validation Conditions:
- If join produces no records, log warning and create empty target table
- If Hudi write fails, job must fail and rollback
- If multiple active records exist for same key after processing, job must fail with data integrity error
-

FR-INCR-001: Incremental Processing Based on Customer Updates

- Title: Incremental SCD2 Update Triggered by Customer Changes
- Description:

The system shall detect changes in customer data and trigger incremental updates to the ordersummary table, applying SCD Type 2 logic only to affected records.

- Business Objective:

Optimize processing performance by updating only changed records rather than full refresh, while maintaining historical accuracy.

- Inputs:
- New/updated customer records (from FR-INGEST-001 and FR-CLEAN-002)
- Existing customer baseline (previous run state)
- Existing ordersummary table (from FR-SCD2-001)

- Processing Rules / Functional Steps:

1. Identify customer records that are new or have changed since last processing

2. For changed customers:

- Retrieve all related order records using CustId
- Apply SCD Type 2 logic (as defined in FR-SCD2-001) to affected ordersummary records

3. Update ordersummary table incrementally using Hudi upsert capabilities

4. Maintain SCD Type 2 columns: IsActive, StartDate, EndDate, OpTs

- Outputs:

- Incrementally updated ordersummary table at s3://adif-sdlc/curated/sdlc_wizard/ordersummary/
- Updated Glue Catalog metadata

- Preconditions:

- Previous run state is available for comparison
- Ordersummary table exists from prior run

- Postconditions:

- Only affected records are updated in ordersummary
- Historical integrity is maintained
- Processing timestamp (OpTs) reflects incremental update time

- Error / Validation Conditions:

- If change detection fails, fall back to full refresh
- If no changes detected, log info message and skip processing
- If incremental update fails, job must fail with error details

-

FR-AGG-001: Generate Customer Aggregate Spend Table

- Title: Customer Aggregate Spend Analytics Table

- Description:

The system shall aggregate order data by customer to calculate total spending and produce an analytics table for business intelligence purposes.

- Business Objective:

Provide summarized customer spending metrics to support business analysis, reporting, and decision-making.

- Inputs:

- Cleaned customer dataset (from FR-CLEAN-002)
- Cleaned order dataset (from FR-CLEAN-002)
- Processing Rules / Functional Steps:
 1. Join customer and order datasets on CustId
 2. Calculate TotalAmount for each order: $\text{TotalAmount} = \text{PricePerUnit Qty}$
 3. Group by customer Name and Date
 4. Aggregate: $\text{sum}(\text{TotalAmount})$ as TotalAmount
 5. Select final columns: Name, TotalAmount, Date
 6. Write results to S3 at `s3://adif-sdlc/analytics/customeragggregatespend/`
 7. Register table in Glue Data Catalog:
 - Database: `gen_ai_poc_databrickscoe`
 - Table Name: `customeragggregatespend`
- Outputs:
 - Analytics table at `s3://adif-sdlc/analytics/customeragggregatespend/`
 - Glue Catalog table: `gen_ai_poc_databrickscoe.customeragggregatespend`
 - Schema: Name, TotalAmount, Date
- Preconditions:
 - Customer and order data are cleaned and deduplicated
 - S3 analytics path is accessible
- Postconditions:
 - Aggregate table is available for querying via Athena
 - Data reflects current state of source datasets
- Error / Validation Conditions:
 - If join produces no records, create empty table and log warning
 - If aggregation fails, job must fail with error details
 - If TotalAmount calculation results in negative values, log warning but continue
-

3. Non-Functional Considerations

Performance

- ETL jobs should leverage Spark parallelism for large datasets
- Incremental processing should significantly reduce runtime compared to full refresh

Scalability

- Solution must handle growing data volumes in source S3 paths
- Hudi table format supports efficient upserts and queries at scale

Monitoring

- AWS CloudWatch should be used to monitor job execution, duration, and errors
- Glue job metrics should be logged for operational visibility

Governance

- AWS Lake Formation may be optionally used for fine-grained access control
- Glue Data Catalog provides centralized metadata management

Reliability

- Failed jobs should not corrupt existing Hudi tables (transactional writes)
- Error handling should provide clear diagnostics for troubleshooting
-

4. Requirement Traceability Notes

Requirement ID	BRD Section	Key Traceability Notes
Req-001	Source Data	Customer Data from S3
Req-002	Source Data	Order Data from S3
Req-003	Data Cleaning	Remove "Null" and NULL values
Req-004	Data Cleaning	Remove duplicates in each dataset
Req-005	Glue Catalog Tables	Register sdwc_wizard_customer in gen_ai_poc_databricksco
Req-006	Glue Catalog Tables	Register sdwc_wizard_order in gen_ai_poc_databricksco
Req-007	SCD Type 2 (Hudi on S3)	Ordersummary table with SCD2 logic, Hudi format at s3://adif-sdlc/curated/sdlc_wizard/ordersummary
Req-008	Incremental Logic	Customer updates trigger SCD2 updates with IsActive, StartDate, EndDate, OpTs
Req-009	Aggregation Table	Customeraggregatespend at s3://adif-sdlc/analytics/customeraggregatespend/ with Name, TotalAm

- End of Functional Requirements Document